

# **PROJET ECOTRACK**

## **ÉTUDE DE FAISABILITÉ**

Master 1 — Filière Développement Logiciel (EADL, RNCP 38822)

## I. CONTEXTE DU PROJET ECOTRACK

✓ Critères RNCP Ce1.1.1 / A1.1 : Vision synthétique des objectifs et contraintes

La métropole, qui regroupe environ 500 000 habitants, assure la collecte des déchets ménagers via un parc de 2 000 conteneurs répartis sur 12 secteurs géographiques. La gestion actuelle repose sur des tournées planifiées à fréquence fixe, sans connaissance du niveau de remplissage réel des conteneurs. Cette approche génère deux dysfonctionnements majeurs : des collectes inutiles sur des conteneurs quasi vides (30 à 40 % des arrêts), et des débordements sur les points les plus sollicités, source de nuisances et de plaintes citoyennes. La plateforme s'adresse à quatre profils d'utilisateurs : 15 000 citoyens actifs, 50 agents de collecte, 10 gestionnaires et 3 administrateurs.

ECOTRACK répond à ce constat par une plateforme IoT de gestion intelligente des déchets. Chaque conteneur est équipé d'un capteur ultrasonique de niveau transmettant une mesure toutes les 15 minutes (ou sur changement significatif) via le protocole MQTT, soit environ 500 000 messages par jour. Ces données alimentent en temps réel une cartographie des niveaux de remplissage et un moteur d'optimisation des tournées fondé sur un algorithme du voyageur de commerce (TSP, raffiné par 2-opt), tenant compte de la capacité des véhicules, des horaires et des zones prioritaires.

La solution s'articule autour de six piliers (connectivité IoT, intelligence artificielle, mobilité, analytics, gamification et sécurité) et se décline en trois applications relevant de la filière Développement : un Dashboard web pour les gestionnaires et administrateurs (monitoring temps réel, création de tournées optimisées, analytics et rapports), une application mobile Citoyen (signalement de conteneurs pleins et gamification) et une application mobile Agent (réception de tournée, scan QR, géolocalisation et mode hors ligne). La gamification — points, badges, défis et leaderboard — constitue un module de développement central destiné à soutenir l'engagement des 15 000 citoyens.

L'objectif est de faire passer la collecte d'une logique calendaire à une logique pilotée par la donnée : ne collecter que les conteneurs qui le nécessitent, au meilleur moment et selon l'itinéraire le plus court. Les cibles visées sont -20 % de distances parcourues, un taux de débordements <2 %, -15 % de coûts opérationnels, -18 % d'émissions

CO<sub>2</sub>, une disponibilité >99,5 % et 15 000 utilisateurs actifs. La présente étude évalue la faisabilité technique, économique et opérationnelle du projet afin de fonder une décision de lancement éclairée.

#### A. Problématique Actuelle

- Collecte inefficace : tournées fixes sans optimisation
- Débordements conteneurs : manque de visibilité temps réel
- Coûts élevés : carburant, main d'œuvre, maintenance
- Impact environnemental : émissions CO<sub>2</sub>, pollution

#### B. Objectifs du Projet

- Réduire les coûts de collecte de 20-30%
- Éliminer les débordements (<5% incidents)
- Optimiser les tournées (algorithme TSP)
- Fournir des analytics temps réel aux gestionnaires

## II. FAISABILITÉ TECHNIQUE

✓ Critères RNCP Ce1.1.2 / A1.3 : Analyse contraintes techniques et faisabilité

### Faisabilité Technique Applicative (Filière Développement)

#### A. Stack Backend

- Framework : Node.js (services temps réel : ingestion IoT, notifications) + FastAPI (Python) pour l'API métier et l'optimisation des tournées
- Base de données : PostgreSQL 15 + extension PostGIS (géospatial : conteneurs, zones, tournées)
- Cache : Redis pour sessions, classements de gamification et requêtes fréquentes (objectif cache hit >80 %)
- Ingestion IoT : capteurs ultrasoniques sur 2 000 conteneurs émettant via MQTT toutes les 15 min (ou sur changement significatif), soit ~500 000 messages/jour, relayés sur un bus Kafka avant persistance

#### B. Stack Frontend

- Framework : React 18 + TypeScript + TailwindCSS (Dashboard web) ; React Native pour les applications mobiles Citoyen et Agent
- Cartographie : Leaflet + OpenStreetMap (états des conteneurs, itinéraires, heatmaps)
- Temps réel : WebSocket (Socket.io) pour push notifications
- Charts : Recharts pour le dashboard analytics (KPIs temps réel, évolutions)

#### C. Algorithme Optimisation Tournées

- TSP (Traveling Salesman Problem) : heuristique gloutonne + raffinement 2-opt pour le MVP, algorithme génétique en évolution
- Contraintes : capacité véhicule, horaires, zones prioritaires
- Performance : calcul tournée optimale <30s pour 50 conteneurs

#### D. Scalabilité et Performance

- Architecture microservices (Domain-Driven Design) derrière une API Gateway : services IoT, Optimisation, API métier, Auth, Notification et Gamification, communiquant via bus Kafka
- Containerisation : Docker + Docker Compose
- Load balancing : Nginx reverse proxy

#### E. Sécurité et Authentification

- API REST documentée (OpenAPI/Swagger), authentification JWT et contrôle d'accès RBAC à 4 rôles (citoyen, agent, gestionnaire, administrateur)

- Mots de passe chiffrés bcrypt (10 rounds), rate limiting 100 req/min/IP, conformité OWASP Top 10, RGPD, WCAG 2.1 AA et ISO 27001

## F. Gamification et Applications Mobiles

- Module de gamification : attribution de points, badges, défis collectifs et leaderboard, exposé via une API dédiée et appuyé sur Redis pour les classements
- Applications mobiles React Native : App Citoyen (signalement + gamification) et App Agent (tourné, scan QR, géolocalisation, mode hors ligne avec synchronisation différée)

## G. Qualité et Industrialisation

- Tests automatisés (unitaires, intégration, e2e) avec couverture >60 % et pipeline CI/CD GitHub Actions (build, test, déploiement conteneurisé Docker)
- Performance cible : API <500 ms (p95), requêtes SQL <200 ms (90 %), chargement page <3 s, 500 utilisateurs simultanés, uptime 99,5 % (RTO <4 h, RPO <1 h)

## III. FAISABILITÉ ÉCONOMIQUE

✓ Critères RNCP Ce1.1.3 / A1.3 : Analyse faisabilité économique et ROI

### A. Coûts de Développement (Année 1)

#### 1. Ressources Humaines (16 semaines = 4 mois)

- Product Owner (25 % temps) : 7 k€
- Scrum Master (25 % temps) : 7 k€
- Développeurs full-stack (3 × 100 %) : 72 k€
- Architecte logiciel (50 % temps) : 16 k€
- DevOps/SRE (50 % temps) : 14 k€

SOUS-TOTAL RH : 116 k€

#### 2. Infrastructure Cloud (4 mois développement)

- Serveurs (3 VM t3.medium AWS/Azure) : 1,5 k€
- Base de données (RDS PostgreSQL) : 1 k€
- Stockage (S3/Blob 100 Go) : 0,2 k€
- Bande passante : 0,3 k€

SOUS-TOTAL Infrastructure Dev : 3 k€

#### 3. Licences et Outils

- Jira (5 users × 4 mois) : 0,2 k€
- GitHub Team (5 users) : 0,1 k€
- Monitoring (Datadog) : 1 k€

- Licences diverses (IDE, design, etc.) : 0,7 k€

SOUS-TOTAL Licences : 2 k€

---

TOTAL ANNÉE 1 (Développement) : 121 k€

## B. Coûts Récurrents (Années 2-3)

### 1. Infrastructure Production (par an)

- Hébergement cloud (production + staging) : 12 k€/an
- Base de données : 4 k€/an
- CDN (CloudFlare) : 1 k€/an
- Backups et stockage : 1 k€/an

SOUS-TOTAL Infrastructure Prod : 18 k€/an

### 2. Maintenance et Support (par an)

- Équipe support (2 personnes 50 %) : 72 k€/an
- Évolutions fonctionnelles : 15 k€/an
- Corrections bugs : 5 k€/an
- Mises à jour sécurité : 5 k€/an

SOUS-TOTAL Maintenance : 97 k€/an

### 3. Licences Récurrentes (par an)

- Outils monitoring/alerting : 3 k€/an
- Licences SaaS : 2 k€/an

SOUS-TOTAL Licences : 5 k€/an

---

TOTAL PAR AN (Années 2-3) : 120 k€/an

### C. TCO (Total Cost of Ownership) 3 ans

- Année 1 (Développement) : 121 k€
  - Année 2 (Production) : 120 k€
  - Année 3 (Production) : 120 k€
- 

—  
TCO 3 ANS : 361 k€

### D. Économies Estimées

Économies annuelles attendues :

- Optimisation des tournées : -20 % de distances parcourues (carburant + usure véhicules) = 90 k€/an
  - Réduction des débordements à <2 % : moins d'interventions d'urgence = 50 k€/an
  - Gains de productivité : -15 % de coûts opérationnels (main-d'œuvre mieux pilotée) = 60 k€/an
  - Bénéfices environnementaux et image : -18 % d'émissions CO<sub>2</sub>, baisse des plaintes citoyennes = 20 k€/an
- 

—  
TOTAL ÉCONOMIES : 220 k€/an

### E. Calcul du ROI

Formule :  $ROI = (Gains\ cumulés - Investissement) / Investissement \times 100$

- Investissement initial (Année 1) : 121 k€
- Économies cumulées sur 3 ans :  $220\text{ k€/an} \times 3 = 660\text{ k€}$
- Coûts récurrents 2 ans :  $120\text{ k€/an} \times 2 = 240\text{ k€}$
- Gain net 3 ans :  $660\text{ k€} - 361\text{ k€} = 299\text{ k€}$

ROI 3 ANS : 83 %

Seuil de rentabilité atteint : 2,1 ans



## IV. FAISABILITÉ OPÉRATIONNELLE

### A. Organisation de l'Équipe Projet

#### 1. Composition de l'Équipe

- Product Owner : 1 personne (profil métier gestion déchets)
  - Scrum Master : 1 personne (certification PSM I)
  - Développeurs Full-Stack : 3 personnes (React + Node.js/Python)
  - Architecte Logiciel : 1 personne (expérience microservices)
  - DevOps/SRE : 1 personne (Docker, Kubernetes, CI/CD)
- TOTAL : 7 personnes (5.5 ETP)

#### 2. Répartition des Rôles et Responsabilités

- PO : Backlog, user stories, validation métier
- SM : Animation Scrum, levée blocages, amélioration continue
- Devs : Développement features, tests, code reviews
- Archi : Conception architecture, choix techniques, POCs
- DevOps : CI/CD, infra, monitoring, déploiements

### B. Méthodologie de Gestion de Projet

- Framework : Scrum (sprints 2 semaines)
- Durée projet : 16 semaines sur 4 mois (Kick-off S1, Sprint 0 S2-3, puis sprints de 2 semaines)
- Rituels : Daily (15 min), Planning (2h), Review (1h), Rétro (1h)
- Outils : Jira + Confluence + Slack + GitHub
- Vitesse cible :  $25 \pm 3$  Story Points par sprint

### C. Planning Prévisionnel

Phase 1 (S1-S7) : Cadrage, socle technique et modules core

- Kick-off (S1) + Sprint 0 (S2-3) : équipes, backlog, architecture microservices, environnement et CI/CD
- Sprint 1 (S4-5) : ingestion MQTT/Kafka, modèle de données PostGIS, API REST + Auth JWT/RBAC
- Sprint 2 (S6-7) : dashboard carte temps réel, alertes, tests de charge

Phase 2 (S8-S16) : optimisation, gamification, mobile et finalisation

- Sprints 3-4 (S8-11) : optimisation TSP+2-opt, dashboard analytics, module gamification, apps mobiles Citoyen et Agent
- Sprint 5 (S12-14) : intégration, tests e2e (coverage >60 %), déploiement, polish
- Soutenance (S15-16) : présentation finale et livraison du DCT (30-35 pages)

### D. Risques Opérationnels

1. Départ membre clé (Prob: Faible, Impact: Élevé)  
Mitigation : Documentation continue, pair programming, knowledge sharing
2. Retard développement fonctionnalité critique (Prob: Moyenne, Impact: Élevé)  
Mitigation : MVP simplifié, priorisation MoSCoW, buffer 10% sur planning
3. Résistance utilisateurs finaux (Prob: Moyenne, Impact: Moyen)  
Mitigation : Formation, beta testing avec chauffeurs, feedback loops
4. Intégration SI existant complexe (Prob: Élevée, Impact: Moyen)  
Mitigation : APIs bien documentées, adapters, tests intégration
5. Dépassement budget (Prob: Moyenne, Impact: Élevé)  
Mitigation : Suivi hebdomadaire budget, alertes à 80%, réserve 10%
6. Indisponibilité de la plateforme cloud (Prob : Faible, Impact : Élevé)  
Mitigation : architecture multi-instances containerisée, supervision proactive (alertes), procédure de reprise documentée et sauvegardes régulières.
7. Non-conformité RGPD des données de géolocalisation (Prob : Moyenne, Impact : Élevé)  
Mitigation : privacy by design, minimisation et chiffrement des données personnelles, registre de traitement et audit de conformité avant mise en production.

## E. Facteurs Clés de Succès

- ✓ Engagement fort du sponsor projet (Directeur Gestion Déchets)
- ✓ Équipe expérimentée et motivée
- ✓ Méthodologie Agile bien maîtrisée
- ✓ Implication utilisateurs finaux dès Sprint 2
- ✓ Architecture scalable et évolutive
- ✓ Tests et validation continue
- ✓ Communication transparente avec parties prenantes

## V. CONCLUSION ET RECOMMANDATIONS

### A. Synthèse de la Faisabilité

#### DÉCISION : GO

##### Faisabilité Technique : FAVORABLE

Stack mature et éprouvée (React 18/React Native, Node.js + FastAPI, PostgreSQL 15 + PostGIS, Redis, Kafka), entièrement open source et déployée en architecture microservices conteneurisée (Docker). Les briques les plus complexes (ingestion MQTT à ~500 000 messages/jour via Kafka, optimisation TSP+2-opt, gamification) sont des problèmes documentés disposant de bibliothèques de référence. Les NFR (API <500 ms p95, 500 users simultanés) sont atteignables, les risques techniques étant atténués par une approche MVP et des tests de charge dès le sprint 3.

##### Faisabilité Économique : FAVORABLE

ROI estimé à 83 % sur 3 ans avec un seuil de rentabilité atteint en 2,1 ans. Les économies annuelles (220 k€/an : -20 % de distances, débordements ramenés à <2 %, -15 % de coûts opérationnels, -18 % de CO<sub>2</sub>) dépassent largement les coûts récurrents (120 k€/an), pour un investissement initial maîtrisé de 121 k€.

##### Faisabilité Opérationnelle : FAVORABLE

Équipe pluridisciplinaire de 7 personnes (5,5 ETP) maîtrisant Scrum et la stack retenue. La méthodologie Agile, l'implication des agents de collecte dès les sprints 3-4 et un plan de gestion des risques structuré sécurisent l'exécution sur les 16 semaines.

#### Conclusion Globale :

L'analyse converge vers une décision de lancement (GO). Sur le plan technique, la solution s'appuie sur des technologies matures et open source organisées en microservices (API Gateway, bus Kafka, PostgreSQL+PostGIS, Redis, Docker), dont la combinaison répond précisément aux contraintes du projet (géospatial, ingestion IoT à ~500 000 messages/jour, temps réel, gamification, applications mobiles). Les exigences de performance (API <500 ms p95, 500 utilisateurs simultanés, uptime 99,5 %) sont réalistes au regard de cette architecture.

Sur le plan économique, le projet est rentable : un investissement initial de 121 k€ génère un ROI de 83 % sur trois ans, avec un retour sur investissement atteint dès la deuxième année grâce à des économies annuelles estimées à 220 k€ (-20 % de distances, débordements <2 %, -15 % de coûts opérationnels, -18 % de CO<sub>2</sub>).

Sur le plan opérationnel, l'équipe et la méthodologie Scrum (sprints de 2 semaines sur 4 mois) sont adaptées au projet. Les principaux points de vigilance — montée en charge de l'ingestion IoT, adoption par les agents de collecte et conformité RGPD — sont identifiés et accompagnés de mesures d'atténuation concrètes.

En conséquence, le projet ECOTRACK est recommandé au lancement, sous réserve du respect d'une approche MVP, d'un suivi budgétaire hebdomadaire et de l'intégration de la sécurité et de l'éco-conception dès la phase de conception.

## B. Recommandations Stratégiques

### 1. Approche MVP (Minimum Viable Product)

- Lancer avec fonctionnalités essentielles (cartographie + alertes + tournées simplifiées)
- Itérer rapidement basé sur feedback utilisateurs réels
- Éviter le perfectionnisme qui retarde la mise en production

### 2. Implication Continue des Utilisateurs Finaux

- Beta testing dès Sprint 4 avec 10 chauffeurs pilotes
- Feedback loops hebdomadaires
- Formation progressive avant déploiement général

### 3. Architecture Scalable dès le Début

- Microservices + containerisation pour faciliter évolutions futures
- Load testing dès Sprint 3 pour anticiper problèmes performance
- Design for failure (résilience, monitoring, alertes)

### 4. Sécurité et Conformité RGPD Natives

- Privacy by design dès conception
- Chiffrement données personnelles (chauffeurs, citoyens)
- Audit sécurité externe avant mise en production

### 5. Documentation Technique Continue

- Architecture Decision Records (ADR) pour choix techniques
- README.md à jour + documentation API (Swagger/OpenAPI)
- Runbooks pour incidents courants

### 6. Tests Automatisés Dès Sprint 1

- Couverture tests unitaires  $\geq 80\%$
- Tests intégration + E2E pour parcours critiques
- CI/CD avec validation automatique avant merge

### 7. Plan de Communication Proactif

- Rapport hebdomadaire aux sponsors (KPIs, risques, décisions)
- Démonstrations sprint toutes les 2 semaines
- Transparence totale sur difficultés rencontrées

---

**FIN DE L'ÉTUDE DE FAISABILITÉ**

---